What's new

Lean Grimoire

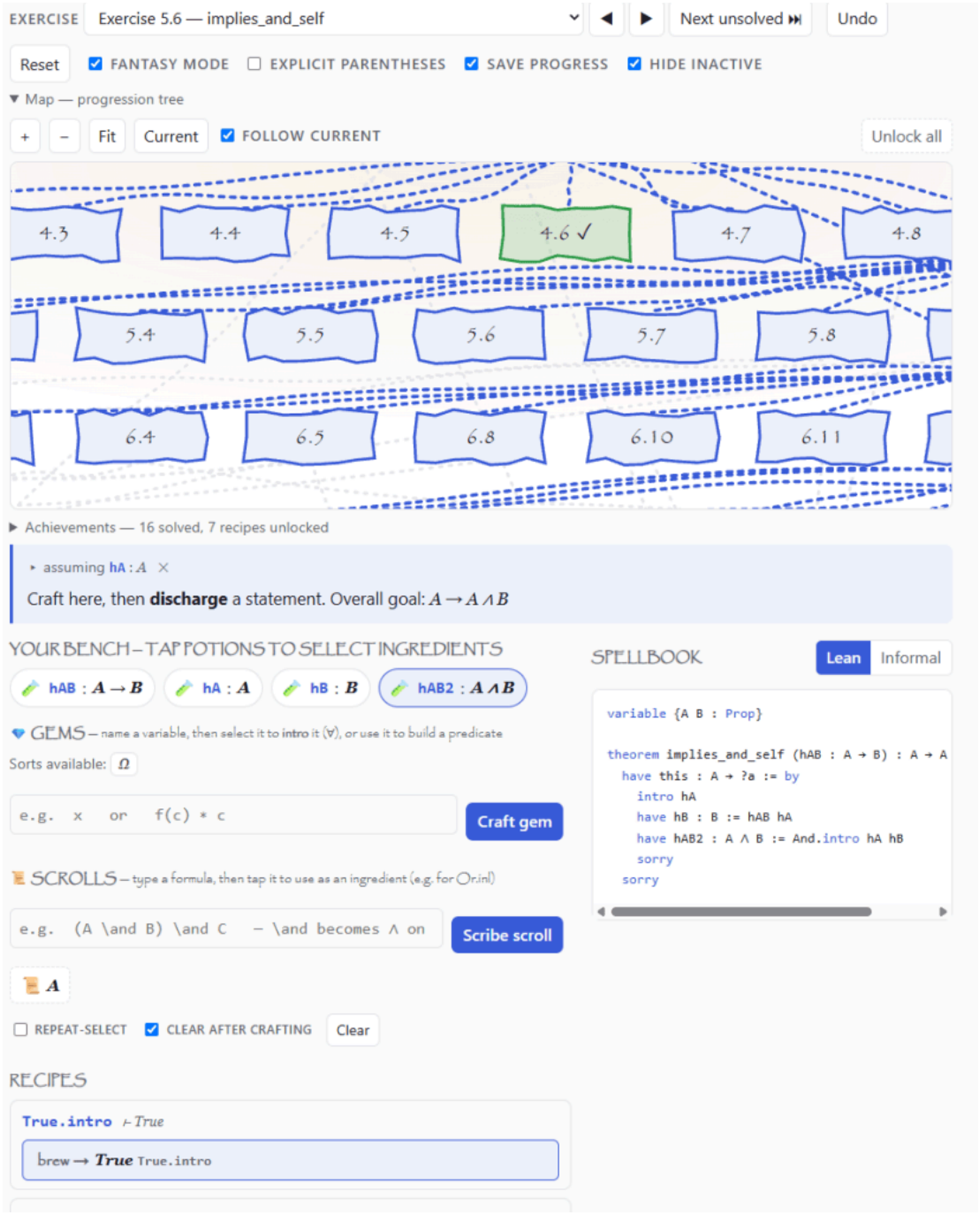
Terence Tao · 11 November 2026

https://terrytao.wordpress.com/2026/11/11/lean-grimoire/

tao2typ · converted to Typst 2026-11-14 (UTC)

Back in July, I started experimenting with using modern coding agents to build a number of mathematical visualization applets. With the extremely hectic events of the last few months, I had to set those projects aside; but I recently found a bit of time to work on one such project that I had started back then, which was to modernize a Javascript interactive textbook I had made back in 2018, entitled "QED", and announced previously on this blog. The purpose of that textbook was to teach propositional and first-order logic in the form of a "crafting game", where one applied various rules of logic to reach a desired conclusion from a given set of hypotheses.

With a coding agent, I rebuilt the game (which I have given the working title of "Lean Grimoire") with a number of new features, for instance giving the option to show an in-progress Lean proof of one's game state. Just for fun, I also added an option to play in a "fantasy mode" in which various game elements are depicted using fantasy-themed flavor text (for instance, mathematical statements become "potions", mathematical expressions become "scrolls", and so forth). Here is a typical screenshot, in which one has nearly completed the proof of the assertion $A \implies A \wedge B$ given $A \implies B$:



I would be interested in having some playtesting of this app to make it more useful as a means to teach logical proofs and/or Lean.

Comments (31)

old-bielefelder · 11 November 2026

A question for clarification: By “game” you mean “deterministic 1-player game”? In that case it might also be called “crafting puzzle” to avoid confusion.

Anonymous · 11 November 2026

A implies A and B?

[Oops, forgot to mention that $A \implies B$ is given as a hypothesis for this particular level -T.]

Anonymous · 11 November 2026

I played several of the levels. I think the intro tactic is confusing for players who are just being introduced to it as a game, without prior exposure to this sort of tactic in a theorem prover.

In my opinion, if you want to make it engaging and understandable for people who don't already understand the material on some level, I would really lean into the visualization. Things should fit together and break apart in a visual/animated way. I am thinking of something like Opus Magnum, if you have seen that game.

I might treat implications as "recipes", when you start an intro tactic, it zooms into a scroll depicting a world with the introduced variables added. Then when you are done it zooms back out to show that scroll existing in the real world as an implication. You could show this zoom in and out recursively.

Just my two cents.

> **Terence Tao** · 11 November 2026
>
> Thanks for the feedback! I had been following reasonably closely the interface of my previously (manually coded) interactive textbook, which was limited by my ability to work with complex graphics, but I am now realizing that with modern coding agents I can indeed implement more sophisticated animations. I got my agent to create a mockup of how one might animate a level that involves `intro`, such as $A \implies A \wedge A$, by creating a "scroll" one zooms into and out of as you suggest: https://claude.ai/artifact/8etb33pyK7MFbX4vPsKDMw . If this seems like a significantly more conceptual type of interface to navigate, I can try to develop it further.
>
> p.s. my agent also pointed me to the game "Patrick's Parabox" that has a similar "zoom in / zoom out" mechanic.
>
> > **old-bielefelder** · 11 November 2026
> >
> > "Patrick's Parabox" is interesting. It reminds me of "Sokoban". there is complexity analysis (NP-hard; PSpace-complete) back in the late 1990s.
> >
> > > **enchantingsecretlyfc1dc3c400** · 12 November 2026
> > >
> > > Sokoban! Great game

Shawn Hood · 11 November 2026

I'm not familiar with Lean myself, but have been armchair following some various discussions in mathematical discipline due to the LLM action of late. My son and I would love to play test the Fantasy version and happy to share feedback

neunhoef · 11 November 2026

I would recommend to switch on "Explicit Parantheses" by default in the beginning, because then one does not have to know that conjunction binds right associatively in Lean. I got confused for a moment by this.

[Now implemented – T.]

Matt · 12 November 2026

I love the idea, but perhaps the logic notation (A ^ B) needs more expansion than the logic terms (hypothesis, statements)? The informal presentation of the steps is excellent, and if you had similarly informal ways of expressing the goal and the steps you could take to get there you might connect with a more general audience of players (and from there tempt them into learning the more technical notation).

> **Terence Tao** · 12 November 2026
>
> Thanks for the suggestion! There is now a “Plain English” version for the logical expressions.

Timothy Daly · 12 November 2026

You might want to look at Patrick Massot’s graph of the LEAN theorems.

I show those that need to be proven. Your game might be an interesting

student guidance project, especially if they use an LLM to help. The

students could be proving real theorems while learning

Anonymous · 12 November 2026

Check out the comments on Hacker News: https://news.ycombinator.com/item?id=50462375

Giovanni Barbarino · 12 November 2026

Maybe add a “Hint” system? Even knowing istintively why a proposition is true, it is not easy to translate it into Lean procedures.

On the other hand, you can run an agent and add for each problem what is the minimum possible number of steps for any given problem given the available resources/recipes at the moment. This would add to the challenge for already experienced users that are not satisfied with longer solutions.

It has always been a dream of mine programming an actual videogame on math, but I barely can scrape time for anything, and not having access to any agent makes it borderline infeasible

> **Terence Tao** · 12 November 2026
>
> Thanks for the suggestions! Hints have now been added for each level, as well as an explicit count of steps compared against the shortest known solution.

Giovanni Barbarino · 12 November 2026

and I just noticed 7.12 is used in previous ones like 7.2

> **Giovanni Barbarino** · 12 November 2026
>
> and also I don’t see how to do 13.4 without knowing Pick that is explained later in 14.1

lovinged57041eed · 12 November 2026

Hello Terence.Thank you for sharing your conceptual scheme for “Lean Grimoire”. The name resonates right away.Initial thoughts and observations:- having a conformal analog, a locally “familiar” model,

Anonymous · 12 November 2026

This is a very nice idea! Also, all good games for developing visualization, better logical reasoning (like finding reasoning flaws, for example), detecting patterns etc are all very welcome!

Anonymous · 12 November 2026

Something that I was telling my students that they have to battle when writing by hand, but they won't face in the game, is malformed expressions. In the game they could combine recipes and get the wrong goal due to, say parentheses in places that are not those of the real goal. However, I didn't see a way to obtain malformed expressions.

Maybe there could be a version of the grimoire that was scribbled over by a wizard apprentice and some recipes are malformed, perhaps deceptively close to other recipes that are well formed. The user still has to use the right ones. Or perhaps, both get to the goal and delete the malformed recipes.

Anonymous · 12 November 2026

Shorter proof for 4.8:

— Lean Grimoire — exercise 4.8 (And.symm')
— https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
— Proved in 3 moves. The shortest proof the game knows takes 5. THIS ONE IS SHORTER.

— We wish to prove (A ∧ B) ⟶ (B ∧ A).
— Introduce A ∧ B:
— From A ∧ B we deduce B ∧ A (the lemma And.comm').
— Discharging, we obtain (A ∧ B) ⟶ (B ∧ A).
— This is what we wished to prove. ∎

— Solving a lemma mints it as a recipe, so a proof that uses one needs that proof with
— it. 1 of them, each as you proved it, deepest first.

— 2.1 (And.comm')
section
variable {A B : Prop}

theorem And.comm' (hAB : A ∧ B) : B ∧ A := by
have hA : A := And.left hAB
have hB : B := And.right hAB
have hBA : B ∧ A := And.intro hB hA
exact hBA
end

— and the proof itself:

variable {A B : Prop}

theorem And.symm' : (A ∧ B) ⟶ (B ∧ A) := by
have hABBA : (A ∧ B) ⟶ (B ∧ A) := by
intro hAB

have hBA : B ∧ A := And.comm' hAB
exact hBA
exact hABBA

Also, there is a somewhat similar game if anyone is interested:

https://adam.math.hhu.de/#/g/leanprover-community/nng4



Terence Tao · 12 November 2026

That works! I've updated the game to record this new proof.



Giovanni Barbarino · 12 November 2026

and it would also be very helpful to know how to write statements with forall and exists in the scrolls.... it does not accept \exists or ∃ or any other variant that I can think of



Anonymous · 12 November 2026

Can you make a linear logic dlc?



PistachioCake · 12 November 2026

A shorter proof for Exercise 2.3!

```
-- Lean Grimoire — exercise 2.3  (And.assoc_right)
-- https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
-- Proved in 5 moves.  The shortest proof the game knows takes 6.  THIS ONE
IS SHORTER.
--
-- We are given A ∧ (B ∧ C).
-- We wish to prove (A ∧ B) ∧ C.
-- From A ∧ (B ∧ C) we deduce (B ∧ C) ∧ A  (the lemma And.comm').
-- From (B ∧ C) ∧ A we deduce B ∧ (C ∧ A)  (the lemma And.assoc_left).
-- From B ∧ (C ∧ A) we deduce (C ∧ A) ∧ B  (the lemma And.comm').
-- From (C ∧ A) ∧ B we deduce C ∧ (A ∧ B)  (the lemma And.assoc_left).
-- From C ∧ (A ∧ B) we deduce (A ∧ B) ∧ C  (the lemma And.comm').
-- This is what we wished to prove.  ∎

-- Solving a lemma mints it as a recipe, so a proof that uses one needs that
proof with
-- it. 2 of them, each as you proved it, deepest first.

-- 2.1  (And.comm')
section
variable {A B : Prop}

theorem And.comm' (hAB : A ∧ B) : B ∧ A := by
  have hA : A := And.left hAB
  have hB : B := And.right hAB
  have hBA : B ∧ A := And.intro hB hA
  exact hBA
end

-- 2.2  (And.assoc_left)
section
variable {A B C : Prop}
```

```
theorem And.assoc_left (hABC : (A ∧ B) ∧ C) : A ∧ (B ∧ C) := by
  have hAB : A ∧ B := And.left hABC
  have hA : A := And.left hAB
  have hB : B := And.right hAB
  have hC : C := And.right hABC
  have hBC : B ∧ C := And.intro hB hC
  have hABC2 : A ∧ (B ∧ C) := And.intro hA hBC
  exact hABC2
end

-- and the proof itself:

variable {A B C : Prop}

theorem And.assoc_right (hABC : A ∧ (B ∧ C)) : (A ∧ B) ∧ C := by
  have hBCA : (B ∧ C) ∧ A := And.comm' hABC
  have hBCA2 : B ∧ (C ∧ A) := And.assoc_left hBCA
  have hCAB : (C ∧ A) ∧ B := And.comm' hBCA2
  have hCAB2 : C ∧ (A ∧ B) := And.assoc_left hCAB
  have hABC2 : (A ∧ B) ∧ C := And.comm' hCAB2
  exact hABC2
```

[Proofs added, thanks – T.]

Anonymous · 12 November 2026

one step proof for Exercise 4.3:

— Lean Grimoire — exercise 4.3
— https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
— Proved in 1 move. The shortest proof the game knows takes 2. THIS ONE IS SHORTER.

— We wish to prove ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C).
— From (A ∧ B) ∨ C we deduce ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) (the lemma Impl.idem).
— This is what we wished to prove. ∎

— Solving a lemma mints it as a recipe, so a proof that uses one needs that proof with
— it. 1 of them, each as you proved it, deepest first.

— 4.1 (Impl.idem)
section
variable {A : Prop}

theorem Impl.idem : A ⟶ A := by
have hAA : A ⟶ A := by
intro hA
exact hA
exact hAA
end

— and the proof itself:

variable {A B C : Prop}

example : ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) := by
have hABCABC : ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) := Impl.idem
exact hABCABC

[Proofs added, thanks – T.]



Anonymous · 12 November 2026

— Lean Grimoire — exercise 6.8 (Or.assoc_left)
— https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
— Proved in 14 moves. The shortest proof the game knows takes 17. THIS ONE IS SHORTER.

— We are given (A ∨ B) ∨ C.
— We wish to prove A ∨ (B ∨ C).
— Introduce A:
— From A, B ∨ C we deduce A ∨ (B ∨ C) (disjunction introduction (left)).
— Discharging, we obtain A ⟶ (A ∨ (B ∨ C)).
— Introduce B:
— From B, C we deduce B ∨ C (disjunction introduction (left)).
— From B ∨ C, A we deduce A ∨ (B ∨ C) (disjunction introduction (right)).
— Discharging, we obtain B ⟶ (A ∨ (B ∨ C)).
— Introduce C:
— From C, B we deduce B ∨ C (disjunction introduction (right)).
— From B ∨ C, A we deduce A ∨ (B ∨ C) (disjunction introduction (right)).
— Discharging, we obtain C ⟶ (A ∨ (B ∨ C)).
— From A ⟶ (A ∨ (B ∨ C)), B ⟶ (A ∨ (B ∨ C)) we deduce (A ∨ B) ⟶ (A ∨ (B ∨ C)) (case analysis (disjunction elimination)).
— From (A ∨ B) ⟶ (A ∨ (B ∨ C)), C ⟶ (A ∨ (B ∨ C)) we deduce ((A ∨ B) ∨ C) ⟶ (A ∨ (B ∨ C)) (case analysis (disjunction elimination)).
— From ((A ∨ B) ∨ C) ⟶ (A ∨ (B ∨ C)), (A ∨ B) ∨ C we deduce A ∨ (B ∨ C) (modus ponens).
— This is what we wished to prove. ∎

variable {A B C : Prop}

theorem Or.assoc_left (hABC : (A ∨ B) ∨ C) : A ∨ (B ∨ C) := by
have hAABC : A ⟶ (A ∨ (B ∨ C)) := by
intro hA
have hABC2 : A ∨ (B ∨ C) := Or.inl hA
exact hABC2
have hBABC : B ⟶ (A ∨ (B ∨ C)) := by
intro hB
have hBC : B ∨ C := Or.inl hB
have hABC2 : A ∨ (B ∨ C) := Or.inr hBC
exact hABC2
have hCABC : C ⟶ (A ∨ (B ∨ C)) := by
intro hC
have hBC : B ∨ C := Or.inr hC
have hABC2 : A ∨ (B ∨ C) := Or.inr hBC
exact hABC2

```
have hABABC : (A ∨ B) → (A ∨ (B ∨ C)) := fun h => Or.elim h hAABC hBABC
have hABCABC : ((A ∨ B) ∨ C) → (A ∨ (B ∨ C)) := fun h => Or.elim h hABABC hCABC
have hABC2 : A ∨ (B ∨ C) := hABCABC hABC
exact hABC2
```

[Proofs added, thanks – T.]

 **Anonymous** · 12 November 2026

```
— Lean Grimoire — exercise 9.7 (not_or_of_imp')
— https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
— Proved in 3 moves. The shortest proof the game knows takes 8. THIS ONE IS SHORTER.

— We are given A → B.
— We wish to prove (¬ A) ∨ B.
— From A we deduce A ∨ (¬ A) (the law of the excluded middle).
— From A ∨ (¬ A), A → B we deduce B ∨ (¬ A) (the lemma Or.imp_left').
— From B ∨ (¬ A) we deduce (¬ A) ∨ B (the lemma Or.comm').
— This is what we wished to prove. ∎

— Solving a lemma mints it as a recipe, so a proof that uses one needs that proof with
— it. 5 of them, each as you proved it, deepest first.

— 4.1 (Impl.idem)
section
variable {A : Prop}

theorem Impl.idem : A → A := by
have hAA : A → A := by
intro hA
exact hA
exact hAA
end

— 6.5 (Or.imp')
section
variable {A B C D : Prop}

theorem Or.imp' (hAB : A ∨ B) (hAC : A → C) (hBD : B → D) : C ∨ D := by
have hACD : A → (C ∨ D) := by
intro hA
have hC : C := hAC hA
have hCD : C ∨ D := Or.inl hC
exact hCD
have hBCD : B → (C ∨ D) := by
intro hB
have hD : D := hBD hB
have hCD : C ∨ D := Or.inr hD
exact hCD
have hABCD : (A ∨ B) → (C ∨ D) := fun h => Or.elim h hACD hBCD
have hCD : C ∨ D := hABCD hAB
```

```
exact hCD
end

— 6.6 (Or.imp_left')
section
variable {A B C : Prop}

theorem Or.imp_left' (hAB : A ∨ B) (hAC : A → C) : C ∨ B := by
have hBB : B → B := Impl.idem
have hCB : C ∨ B := Or.imp' hAB hAC hBB
exact hCB
end

— 6.2 (Or.symm')
section
variable {A B : Prop}

theorem Or.symm' : (A ∨ B) → (B ∨ A) := by
have hABA : A → (B ∨ A) := by
intro hA
have hBA : B ∨ A := Or.inr hA
exact hBA
have hBBA : B → (B ∨ A) := by
intro hB
have hBA : B ∨ A := Or.inl hB
exact hBA
have hABBA : (A ∨ B) → (B ∨ A) := fun h => Or.elim h hABA hBBA
exact hABBA
end

— 6.3 (Or.comm')
section
variable {A B : Prop}

theorem Or.comm' (hAB : A ∨ B) : B ∨ A := by
have hABBA : (A ∨ B) → (B ∨ A) := Or.symm'
have hBA : B ∨ A := hABBA hAB
exact hBA
end

— and the proof itself:

variable {A B : Prop}

theorem not_or_of_imp' (hAB : A → B) : (¬ A) ∨ B := by
have hAA : A ∨ (¬ A) := Classical.em (A)
have hBA : B ∨ (¬ A) := Or.imp_left' hAA hAB
have hAB2 : (¬ A) ∨ B := Or.comm' hBA
exact hAB2
```

[Proofs added, thanks – T.]

Jacob · 13 November 2026

Shorter proof for 4.3:

— Lean Grimoire — exercise 4.3
— https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
— Proved in 1 move. The shortest proof the game knows takes 2. THIS ONE IS SHORTER.

— We wish to prove ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C).
— From (A ∧ B) ∨ C we deduce ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) (the lemma Impl.idem).
— This is what we wished to prove. ∎

— Solving a lemma mints it as a recipe, so a proof that uses one needs that proof with
— it. 1 of them, each as you proved it, deepest first.

— 4.1 (Impl.idem)
section
variable {A : Prop}

theorem Impl.idem : A ⟶ A := by
have hAA : A ⟶ A := by
intro hA
exact hA
exact hAA
end

— and the proof itself:

variable {A B C : Prop}

example : ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) := by
have hABCABC : ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) := Impl.idem
exact hABCABC

[Proofs added, thanks – T.]

Dario Bajo · 13 November 2026

Exercise 4.3

QED! You brewed *((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C)* in 1 move. 🏆 **A new record.** The shortest proof this game knows takes 2 moves, so you have found something it does not. Please tell me about it: **Copy** or **Save** the proof below and leave it in the comments on the blog post.

— Lean Grimoire — exercise 4.3
— https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
— Proved in 1 move. The shortest proof the game knows takes 2. THIS ONE IS SHORTER.

— We wish to prove ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C).
— From (A ∧ B) ∨ C we deduce ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) (the lemma Impl.idem).
— This is what we wished to prove. ∎

— Solving a lemma mints it as a recipe, so a proof that uses one needs that proof with
— it. 1 of them, each as you proved it, deepest first.

— 4.1 (Impl.idem)
section
variable {A : Prop}

theorem Impl.idem : A ⟶ A := by
have hAA : A ⟶ A := by
intro hA
exact hA
exact hAA
end

— and the proof itself:

variable {A B C : Prop}

example : ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) := by
have hABCABC : ((A ∧ B) ∨ C) ⟶ ((A ∧ B) ∨ C) := Impl.idem
exact hABCABC

[Proofs added, thanks – T.]

 **Anonymous** · 13 November 2026

Shorter proof for 2.3 :D
(Tried commenting this earlier, but it didn't show up… If there's a duplicate, sorry!)

— Lean Grimoire — exercise 2.3 (And.assoc_right)
— https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
— Proved in 5 moves. The shortest proof the game knows takes 6. THIS ONE IS SHORTER.

— We are given A ∧ (B ∧ C).
— We wish to prove (A ∧ B) ∧ C.
— From A ∧ (B ∧ C) we deduce (B ∧ C) ∧ A (the lemma And.comm').
— From (B ∧ C) ∧ A we deduce B ∧ (C ∧ A) (the lemma And.assoc_left).
— From B ∧ (C ∧ A) we deduce (C ∧ A) ∧ B (the lemma And.comm').
— From (C ∧ A) ∧ B we deduce C ∧ (A ∧ B) (the lemma And.assoc_left).
— From C ∧ (A ∧ B) we deduce (A ∧ B) ∧ C (the lemma And.comm').
— This is what we wished to prove. ∎

— Solving a lemma mints it as a recipe, so a proof that uses one needs that proof with
— it. 2 of them, each as you proved it, deepest first.

— 2.1 (And.comm')
section
variable {A B : Prop}

theorem And.comm' (hAB : A ∧ B) : B ∧ A := by
have hA : A := And.left hAB
have hB : B := And.right hAB
have hBA : B ∧ A := And.intro hB hA
exact hBA
end

— 2.2 (And.assoc_left)
section
variable {A B C : Prop}

theorem And.assoc_left (hABC : (A ∧ B) ∧ C) : A ∧ (B ∧ C) := by
have hC : C := And.right hABC
have hAB : A ∧ B := And.left hABC
have hA : A := And.left hAB
have hB : B := And.right hAB
have hBC : B ∧ C := And.intro hB hC
have hABC2 : A ∧ (B ∧ C) := And.intro hA hBC
exact hABC2
end

— and the proof itself:

variable {A B C : Prop}

theorem And.assoc_right (hABC : A ∧ (B ∧ C)) : (A ∧ B) ∧ C := by
have hBCA : (B ∧ C) ∧ A := And.comm' hABC
have hBCA2 : B ∧ (C ∧ A) := And.assoc_left hBCA
have hCAB : (C ∧ A) ∧ B := And.comm' hBCA2
have hCAB2 : C ∧ (A ∧ B) := And.assoc_left hCAB
have hABC2 : (A ∧ B) ∧ C := And.comm' hCAB2
exact hABC2

[Proofs added, thanks – T.]

 **Kantian** · 13 November 2026

You advise to post shorter proof if we find one, so here is one

— Lean Grimoire — exercise 6.8 (Or.assoc_left)
— https://terrytao.wordpress.com/2026/11/11/lean-grimoire/
— Proved in 14 moves. The shortest proof the game knows takes 17. THIS ONE IS SHORTER.
—
— We are given (A ∨ B) ∨ C.
— We wish to prove A ∨ (B ∨ C).
— Introduce A:
— From A, B ∨ C we deduce A ∨ (B ∨ C) (disjunction introduction (left)).
— Discharging, we obtain A ⟶ (A ∨ (B ∨ C)).
— Introduce B:
— From B, C we deduce B ∨ C (disjunction introduction (left)).
— From B ∨ C, A we deduce A ∨ (B ∨ C) (disjunction introduction (right)).
— Discharging, we obtain B ⟶ (A ∨ (B ∨ C)).
— From A ⟶ (A ∨ (B ∨ C)), B ⟶ (A ∨ (B ∨ C)) we deduce (A ∨ B) ⟶ (A ∨ (B ∨ C)) (case analysis (disjunction elimination)).
— Introduce C:
— From C, B we deduce B ∨ C (disjunction introduction (right)).
— From B ∨ C, A we deduce A ∨ (B ∨ C) (disjunction introduction (right)).

— Discharging, we obtain C ⟶ (A ∨ (B ∨ C)).
— From (A ∨ B) ⟶ (A ∨ (B ∨ C)), C ⟶ (A ∨ (B ∨ C)) we deduce ((A ∨ B) ∨ C) ⟶ (A ∨ (B ∨ C)) (case analysis (disjunction elimination)).
— From ((A ∨ B) ∨ C) ⟶ (A ∨ (B ∨ C)), (A ∨ B) ∨ C we deduce A ∨ (B ∨ C) (modus ponens).
— This is what we wished to prove. ∎

```
variable {A B C : Prop}

theorem Or.assoc_left (hABC : (A ∨ B) ∨ C) : A ∨ (B ∨ C) := by
 have hAABC : A ⟶ (A ∨ (B ∨ C)) := by
 intro hA
 have hABC2 : A ∨ (B ∨ C) := Or.inl hA
 exact hABC2
 have hBABC : B ⟶ (A ∨ (B ∨ C)) := by
 intro hB
 have hBC : B ∨ C := Or.inl hB
 have hABC2 : A ∨ (B ∨ C) := Or.inr hBC
 exact hABC2
 have hABABC : (A ∨ B) ⟶ (A ∨ (B ∨ C)) := fun h => Or.elim h hAABC hBABC
 have hCABC : C ⟶ (A ∨ (B ∨ C)) := by
 intro hC
 have hBC : B ∨ C := Or.inr hC
 have hABC2 : A ∨ (B ∨ C) := Or.inr hBC
 exact hABC2
 have hABCABC : ((A ∨ B) ∨ C) ⟶ (A ∨ (B ∨ C)) := fun h => Or.elim h hABABC hCABC
 have hABC2 : A ∨ (B ∨ C) := hABCABC hABC
 exact hABC2
```

[Proofs added, thanks – T.]